Advance Mobile Application Development

Local Database

Dr. Mazin Alkathiri
IT Department
Seiyun University
2023-2024

Flutter – Database Concepts

- Flutter provides many advanced packages to work with databases. The most important packages are:
- sqflite – Used to access and manipulate SQLite database, and
- firebase_database – Used to access and manipulate cloud hosted NoSQL database from Google.

SQLite

- SQLite database is the de-facto and standard SQL based embedded database engine.
- It is small and time-tested database engine.
- sqflite package provides a lot of functionality to work efficiently with SQLite database.
- It provides standard methods to manipulate SQLite database engine.
- The core functionality provided by sqflite package is as follows:
  - Create / Open (openDatabase method) a SQLite database.
  - Execute SQL statement (execute method) against SQLite database.
  - Advanced query methods (query method) to reduce to code required to query and get information from SQLite database.

- SQLite doesn't need a server or backend code, all the info is saved to a computer file within the device, or we will say it's native to storage.
- SQLite is used by literally millions of applications with literally billions and billions of deployments. SQLite is the most widely deployed database engine in the world today.

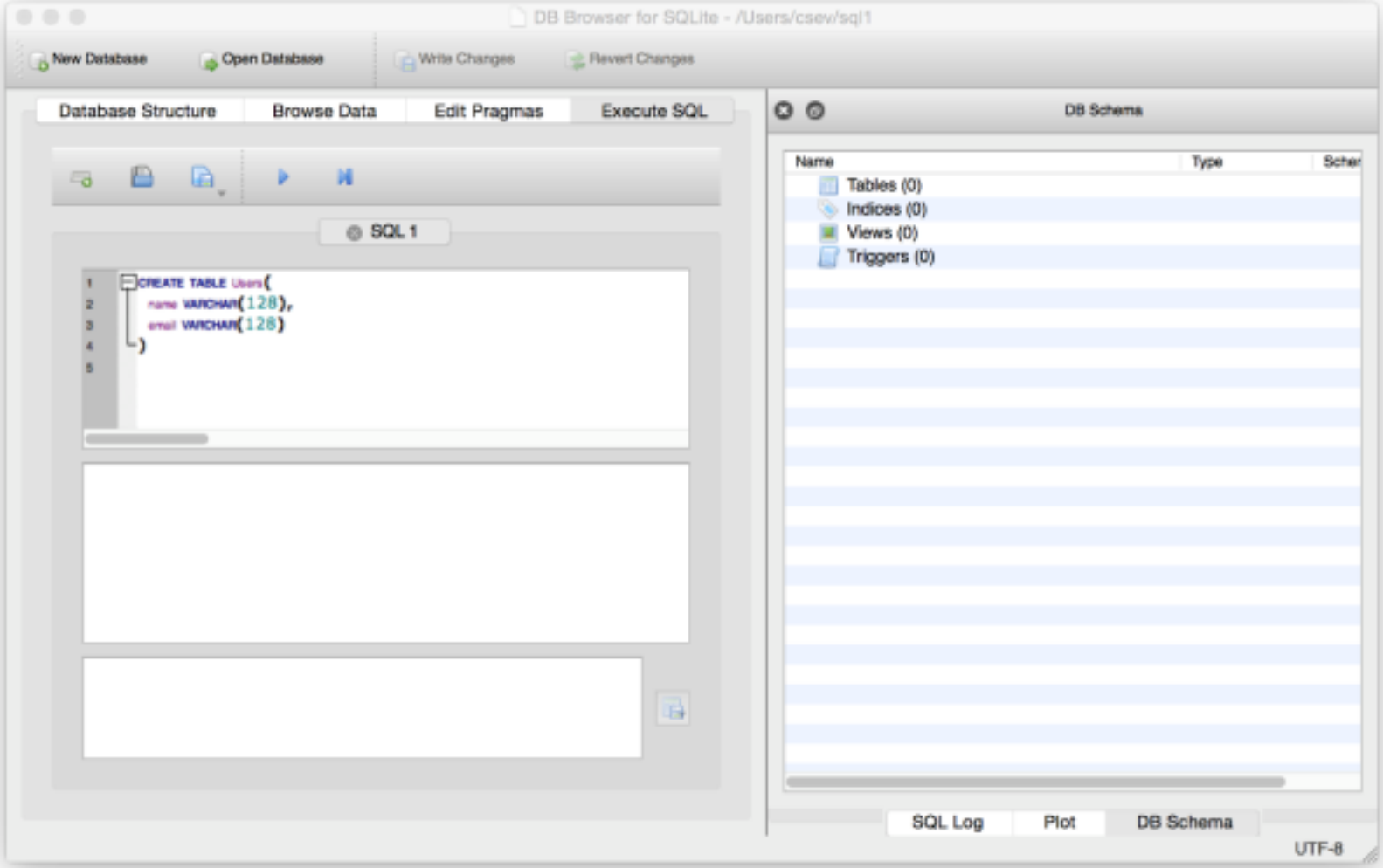
http://www.sqlite.org/famous.html

SQLite Browser

SQLite is a very popular database - it is free and fast and small SQLite Browser allows us to directly manipulate SQLite files

http://sqlitebrowser.org/

SQLite is embedded in Python and a number of other languages




```
CREATE TABLE Users(
  name VARCHAR(128),
  email VARCHAR(128)
)
```

DB Browser for SQLite - /Users/csev/sql1

New Database | Open Database | Write Changes | Revert Changes

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Create Table | Modify Table | Delete Table

Name	Type	Schema
Tables (1)		
Users		CREATE TABLE Users(name VARCHAR(128), email VARCHAR(128))
Indices (0)		
Views (0)		
Triggers (0)		

DB Schema

Name	Type	Schema
Tables (1)		
Users		CREATE TABLE Users(name VARCHAR(128), email VARCHAR(128))
Indices (0)		
Views (0)		
Triggers (0)		

SQL Log | Plot | DB Schema

UTF-8

DB Browser for SQLite - /Users/csev/sql1

New Database | Open Database | Write Changes | Revert Changes

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: Users | New Record | Delete Record

	name	email
	Filter	Filter
1	Chuck	csev@umich...
2	Colleen	cvl@umich.edu
3	Ted	ted@umich....
4	Sally	a1@umich.edu

|< < 0 - 0 of 0 > >| Go to: 1

DB Schema

Name	Type	Schema
Tables (1)		
Users		CREATE TABLE Users(name VARCHAR(128), email VARCHAR(128))
Indices (0)		
Views (0)		
Triggers (0)		

Our table with four rows

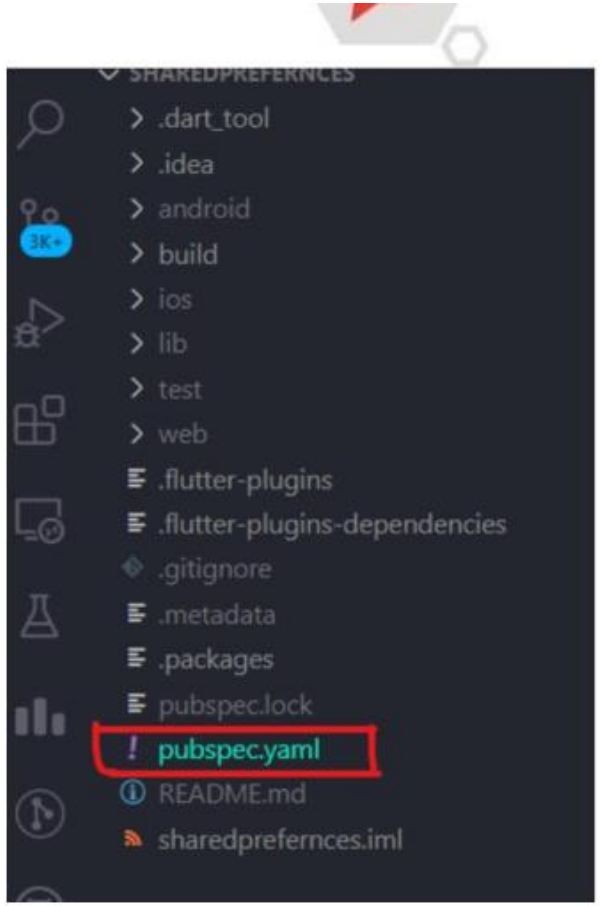
SQL Log | Plot | DB Schema

UTF-8

Installing SQLite Plugin to Flutter

- To be able to use SQLite in Flutter, you wish to feature the plugin SQLite. thus to feature
- it you wish to navigate to the pubspec.yaml file

```
flutter:
  sdk: flutter
path: ^1.8.0
sqflite: ^2.0.0+4
```



>> flutter pub get

README.md | main.dart | pubspec.yaml | sqldb.dart

Visual layout of bidirectional text can depend on the base direction (View | Bidi Text Base Direction)

```
1   // يجب حفظ هذا الملف لانشاء اي قاعدة بيانات
2
3   import 'package:sqflite/sqflite.dart';
4   import 'package:path/path.dart';
5   class SqlDb {
6     // this is made to not make init again and again
7     static Database? _db;
8
9     Future<Database?> get db async {...}
17
18    // here we init the database and creat the tables
19    initalDb() async {...}
26
27    _onUpgrade(Database db, int oldversion, int newversion) async {...}
39
40    _onCreate(Database db, int version) async {...}
56  }
```

Define the database variable DB

```
Future<Database?> get db async {
  if (_db == null) {
    _db = await initalDb();
    return _db;
  } else {
    return _db;
  }
}
```

```
initalDb() async {
  String databasepath = await getDatabasesPath();
  String path = join(databasepath, 'mazdb.db');
  Database mydb = await openDatabase(
      path, onCreate: _onCreate, version: 2, onUpgrade: _onUpgrade);
  return mydb;
}
```

Get the default databases location.
Open the database at a given path

version (optional) specifies the schema version of the database being opened. This is used to decide whether to call onCreate, onUpgrade, and onDowngrade

The optional callbacks are called in the following order:

1. onConfigure
2. onCreate or onUpgrade or onDowngrade
3. onOpen

```
_onCreate(Database db, int version) async {
  await db.execute('''
      CREATE TABLE "notes"(
      "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
      "note" TEXT NOT NULL
      )
      ''');
  await db.execute('''
      CREATE TABLE "users"(
      "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
      "user_name" TEXT NOT NULL,
      "user_password" TEXT NOT NULL
      )
      ''');
  print("======onCreat database and tables ================");
}
```

```
_onUpgrade(Database db, int oldversion, int newversion) async {
  await db.execute('''
      CREATE TABLE "products"(
      "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
      "pro_name" TEXT NOT NULL,
      "pro_type" TEXT NOT NULL,
      "pro_desc" TEXT NOT NULL
      )
      ''');

  print("onUpgrae ==========================");
}
```

```
// SELECT
  readData(String sql) async{...}
// INSERT
  insertData(String sql) async{...}
// UPDATE
  updateData(String sql) async{...}
// DELETE
  deleteData(String sql) async{...}
```

```
// SELECT
  readData(String sql) async{
    Database? mydb = await db;
    List<Map> response = await mydb!.rawQuery(sql);
    return response;
  }
```

```
SqlDb sqlDb=SqlDb();
```

```
child: MaterialButton(
  color: Colors.blue,
  onPressed:() async{
    List<Map> response=  await sqlDb.readData("SELECT * FROM 'notes'");
    print(response);
  },
  child:Text("Read data"),
),
```

```
// INSERT
  insertData(String sql) async{
    Database? mydb = await db;
    int response = await mydb!.rawInsert(sql);
    return response;
  }
```

```
SqlDb sqlDb=SqlDb();
```

```
child: MaterialButton(
  color: Colors.amber,
  onPressed:() async{
    int response = await
            sqlDb.insertData("INSERT INTO 'notes' ('note') VALUES ('my note is ..... some text over here ')");
    print(response);
  },
  child:Text("Insert data"),
),
```

```
// UPDATE
  updateData(String sql) async{
    Database? mydb = await db;
    int response = await mydb!.rawUpdate(sql);
    return response;
  }
```

```
SqlDb sqlDb=SqlDb();
```

```
child: MaterialButton(
  color: Colors.blue,
  onPressed:() async{
    int response=  await sqlDb.updateData("UPDATE 'notes' SET note = 'not two' WHERE id = 2");
    print(response);

  },
  child:Text("update data"),
),
```

```
// DELETE
  deleteData(String sql) async{
    Database? mydb = await db;
    int response = await mydb!.rawDelete(sql);
    return response;
  }
```

```
SqlDb sqlDb=SqlDb();
```

```
child: MaterialButton(
  color: Colors.blue,
  onPressed:() async{
    int response=  await sqlDb.deleteData("DELETE FROM 'notes' WHERE id=1");
    print(response);

  },
  child:Text("Delete data"),
),
```